

Práctico 6 - Técnicas de Mejoras de Rendimiento

AdC - 2025

Ejercicio 1: Deep Pipelines

Considere construir un procesador con pipeline dividiendo el procesador de un solo ciclo en N etapas. El procesador de ciclo único tiene un retardo de propagación a través de la lógica combinacional de 740ps. La penalidad por agregar un registro de pipeline es de 90ps. Suponga que el retardo de la lógica combinacional se puede dividir arbitrariamente en cualquier número de etapas y que la lógica de hazard del pipeline no aumenta el retardo.

Asumiendo que un pipeline de cinco etapas tiene un CPI de 1.23 y que cada etapa adicional aumenta el CPI en 0.1 debido a las predicciones de salto erróneas y otros hazard. ¿Cuántas etapas de pipeline deberían usarse para hacer que el procesador ejecute los programas lo más rápido posible?

$$\text{Tiempo de ciclo} = \left(740/N + 90\right) \text{ps}$$

$$\text{CPI} = 1,23 + 0,1 \cdot (N - 5) \Rightarrow \text{dsq de las 5 etapas, todas sumarán 0,1 al CPI}$$

$$\text{Tiempo por instrucción} = \text{Tiempo de ciclo} \cdot \text{CPI}$$

Etapas	Tiempo por instrucción
5	292,74
6	283,73
7	279,87
8	279,23
9	280,72
10	283,72

} luego de las 8 etapas, el Ti comienza a aumentar

Deberían utilizarse 8 etapas de Pipeline

Ejercicio 2: Predictores de saltos

Asuma un microprocesador con 20 etapas de pipeline, con un fetch que levanta 5 instrucciones por ciclo. Este procesador ejecuta un código donde 1 de cada 5 instrucciones es un salto y esta conformado por bloques de 5 instrucciones donde la última es un salto. La penalidad por una mala predicción es de 20 ciclos. ¿Cuántos ciclos de instrucción toma hacer fetch de todas las instrucciones? Considerando predictores con las siguientes precisiones: 100%, 99%, 90%, 60%.

20 etapas

1 ciclo = 5 instrucciones

Código $\Rightarrow$ 1 de cada 5 $\rightarrow$ salto (al último)

Sup. 500 instrucciones

$500 / 5 \text{ (instrus} \times \text{ ciclo)} \Rightarrow 100 \text{ ciclos}$

Precisión	Ciclos
100 %	100 (s/ errores)
99 %	$100 + 20$ (falla el salto en 1 ciclo de 100)
90 %	$100 + 20 - 10$ (falla el salto en 10 ciclos)
60 %	$100 + 20 \cdot 40$ (falla el salto en 40 ciclos)

Ejercicio 3: Predictores de saltos

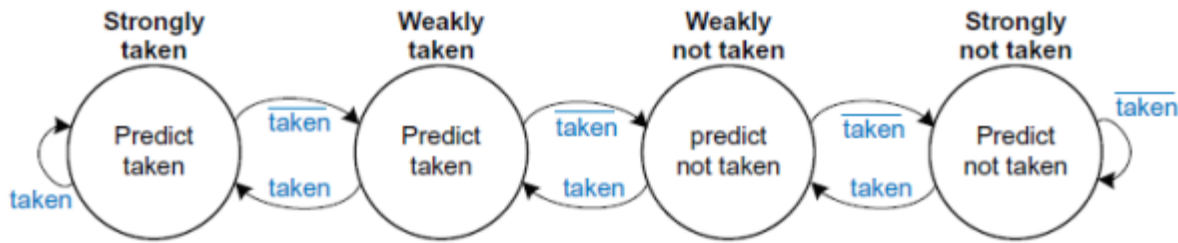


Figure 7.62 Two-bit branch predictor state transition diagram

Este ejercicio analiza la precisión de varios predictores de saltos para el siguiente patrón repetitivo (ej, en un loop) donde los saltos resultaron: **Taken - Not Taken - Taken - Taken - Not Taken**.

- ¿Cuál es la precisión de los predictores *always-taken* y *always-not taken* para el patrón dado?
- ¿Cuál es la precisión del predictor de 2-bits para los primeros 4 saltos de este patrón? Asumir que el predictor arranca en *Strongly not taken*.
- ¿Cuál es la precisión de este predictor de 2-bits si el patrón completo se repite infinitamente?

a- Always-taken: $3/5 = 60\%$ precisión
 Always-not taken: $2/5 = 40\%$ precisión

b-

Salto	T	NT	T	T	NT
Predictor	00	01	00	01	10
Predicción	NT	NT	NT	NT	T

$1/5 = 20\%$ precisión

c-

2ª vez → 3ª vez →

Salto	T	NT	T	T	NT	T	NT	T	T	NT	T	NT	T	T	NT
Predictor	00	01	00	01	10	01	10	01	10	11	10	11	10	11	11
Predicción	NT	NT	NT	NT	T	NT	T	NT	T	T	T	T	T	T	T

5ª vez → 6ª vez →

Salto	T	NT	T	T	NT	T	NT	T	T	NT	T	NT	T	T	NT
Predictor	10	11	10	11	11	10	11	10	11	11	10	11	10	11	11
Predicción	T	T	T	T	T	T	T	T	T	T	T	T	T	T	T

A partir de la 3ª vez, siempre arranco de weakly taken, por lo que la precisión siempre será $3/5 = 60\%$

Ejercicio 4: Predictores de saltos

Asumiendo que la distribución de instrucciones dinámicas se divide en las siguientes categorías:

R-Type	CBZ/CBNZ	B	LDUR	STUR
40%	25%	5%	25%	5%

y las siguientes precisiones en los métodos de predicción de salto:

Always-Taken	Always-Not-Taken	2-Bit
45%	55%	85%

Considerando que el resultado y la dirección del salto se determinan en la etapa de decodificación (ID) y se aplican en la etapa de ejecución (EX) y que no hay hazards de datos.

- a) ¿Cuántos CPI (Ciclos por instrucción) extras se producen debido a los fallos de predicción del método *Always-Taken*?
- b) ¿Cuántos CPI (Ciclos por instrucción) extras se producen debido a los fallos de predicción del método *Always-Not-Taken*?
- c) ¿Cuántos CPI (Ciclos por instrucción) extras se producen debido a los fallos de predicción del método *2-Bit*?

Sabemos que la penalidad por errarle a un salto es de 3 ciclos (en nuestro micro)

a - 45% de precisión $\Rightarrow$ erro el 55% de los saltos

$$0,3 \cdot 0,55 \cdot 3 = 0,495 \Rightarrow \text{casi } 50\% \text{ de CPI extra}$$

b - 55% de precisión $\Rightarrow$ erro 45% de los saltos

$$0,3 \cdot 0,45 \cdot 3 = 0,405 \Rightarrow 40\% \text{ de CPI extra}$$

c - 85% de precisión $\Rightarrow$ erro 15% de los saltos

$$0,3 \cdot 0,15 \cdot 3 = 0,135 \Rightarrow 13,5\% \text{ de CPI extra}$$

Ejercicio 5: Predictores de saltos

El siguiente código en C puede escribirse en ARMv8 de la siguiente forma:

```

for (i = 0; i < 100; i++) {
    for (j = 0; j < 3; j++) {
        ...
    }
}

0x00:      add x0, xzr, xzr
0x04: L2:   add x1, xzr, xzr
0x08: L1:   ...
0x0C:      addi x1, x1, 1
0x10:      cmpi x1, 3
0x14:      b.lt L1
0x18:      addi x0, x0, 1
0x1C:      cmpi x0, 99
0x20:      b.lt L2
    
```

a) Mostrar como queda la tabla de historial de patrones (PHT) considerando que el procesador que ejecuta este código, cuenta con un predictor de saltos local de dos niveles.

b) Comparar la precisión de este predictor con uno de 2-bits (despreciando los primeros ciclos de iniciación).

Si ponemos $n=4$ y $m=4$ n = bits de GR
 m = bits menos significativos

Como el predictor es el que como el PC local, tendrá en cuenta solo el caso anteriormente con ese mismo salto que estoy tratando de predecir

a. Loop j

	GR	Predicción
$j=0$	1110	Taken
$j=1$	1101	Taken
$j=2$	1011	Taken
$j=3$	0111	Not taken

Loop i

$\forall i < 100$
 el patrón será
 T, T, T, ...
 cuando $i = 100$
 el patrón será
 T, T, ..., T, NT
 (1110)

PHT (c/ patrones reconocidos)

GR	PC	Resultado
1110	0100	11
1101	0100	11
1011	0100	11
0111	0100	00
1111	0000	11

loop j,
precisión
100%
loop i
precisión
99%.

b. El patrón repetitivo para loop j es
T T T NT; en un momento,
tengo que mi predictor de 2 bits
estará en 11 al inicio del patrón.

Salto	T	T	T	NT
Predictor	11	11	11	11
Predicción	T	T	T	T

Precisión de $3/4 = 75\%$.

Ejercicio 6: Predictores de saltos

Asuma que el siguiente código itera en un array largo y lleno de números enteros positivos aleatorios.

El código cuenta con 4 saltos, etiquetados B1, B2, B3 y B4. Cuando decimos que un salto es Taken, nos referimos a que el código dentro de las llaves es ejecutado.

```
for (int i=0; i<N; i++) {           /* B1 */
    val = array[i];                 /* TAKEN PATH for B1 */
    if (val % 2 == 0) {             /* B2 */
        sum += val;                 /* TAKEN PATH for B2 */
    }
    if (val % 3 == 0) {             /* B3 */
        sum += val;                 /* TAKEN PATH for B3 */
    }
    if (val % 6 == 0) {             /* B4 */
        sum += val;                 /* TAKEN PATH for B4 */
    }
}
```

a) Determinar cuál de los cuatro saltos muestra una correlación local.

b) ¿Existe correlación global entre algunos de los saltos? Explicar.

a_ B1 tiene correlación local, depende de cuántas veces lo tome antes si voy o no a tomarlo a continuación.

b_ B2, B3 y B4 tienen correlación global. Si entré en B2 y B3 $\Rightarrow$ entro en B4

Si no entré en B2 o en B3 $\Rightarrow$ no entro en B4

Ejercicio 7: Static Multiple Issue Processor

Considere un procesador LEGv8 two-issue, donde en cada "issue packet" una de las instrucciones puede ser una operación de la ALU o un salto y la otra puede ser un *load* o *store*, tal como se muestra en la figura. El compilador asume toda la responsabilidad de eliminar los hazard, organizar el código e insertar operaciones tipo "nop" para que el código se ejecute sin necesidad de detección de hazard o generación de stalls.

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

Considere el siguiente bucle:

```

1  Loop:  LDUR X0, [X20,#0]  // X0=array element
2         ADD X0,X0,X21      // add scalar in X21
3         STUR X0, [X20,#0]  // store result
4         SUBI X20,X20,#8     // decrement pointer
5         CMP X20,X22         // compare to loop limit
6         B.GT Loop          // branch if X20 > X22

```

- Analice en el código las dependencias de datos y determine cuales generan data hazards en nuestro procesador one-issue, **sin** forwarding-stall. En cada caso indique: el tipo de hazard, el operando en conflicto y los números de las instrucciones involucradas. Suponga que los saltos están perfectamente predichos, de modo que los *control hazard* son manejados por hardware.
- ¿Cómo se organizaría el siguiente código en un procesador two-issue con pipeline y forwarding-stall para evitar la mayor cantidad posible de stalls?
- Suponga que el compilador es capaz de determinar que la cantidad de iteraciones del bucle se da en múltiplos de 2. Utilice la técnica estática "*loop unrolling*" para re-ordenar la ejecución del código. Determine mejora en el tiempo de ejecución respecto al punto (b). Se calcula como: *Tiempo de ejecución sin mejora / Tiempo de ejecución con mejora*

a-

Hazard	Operando	Instrucciones
Raw	X0	1-2 ●
Raw	X0	2-3 ●
Raw	X20	4-5 ●
Waw	X0	1-2 ◌

me dio
falta,
hecho en
las
filminas
de clase

b.

ALU / Branch	Load/store
SUBI X20, X20, #8	LDUR X0, [X20, #0]
ADD X0, X0, X21	NOP
CMP X20, X22	STUR X0, [X20, #0]
B.GT loop	NOP

Ejercicio 8: Static Multiple Issue Processor

En este ejercicio se compara el rendimiento de los procesadores de 1-issue y 2-issue, teniendo en cuenta las transformaciones que se pueden realizar en un programa para optimizar la ejecución de 2-issue.

Los problemas en este ejercicio se refieren al siguiente bucle (escrito en C):

```
for(i=0; i!=j; i+=2)
    b[i]=a[i]-a[i+1];
```

El código utiliza los siguientes registros:

i	j	a	b	Temporary values
X5	X6	X1	X2	X10-X15

Un compilador con poca o ninguna optimización podría generar el siguiente código de assembler LEGv8:

```

ADD X5, XZR, XZR
B ENT
TOP:  LSL X10, X5, #3
      ADD X11, X1, X10
      LDUR X12, [X11, #0]
      LDUR X13, [X11, #8]
      SUB X14, X12, X13
      ADD X15, X2, X10
      STUR X14, [X15, #0]
      ADDI X5, X5, #2
ENT:  CMP X5, X6
      B.NE TOP
```

Asumiendo que el procesador de 2-issue tiene las siguientes propiedades:

1. En cada *issue packet* una instrucción debe ser una operación de memoria y la otra una de tipo aritmética/lógica o un salto.
2. El procesador tiene todos los caminos de forwarding posibles entre las etapas (incluyendo los caminos a la etapa ID para la resolución de saltos).
3. El procesador predice los saltos perfectamente.
4. Dos instrucciones no pueden procesarse juntas en un paquete si una depende de la otra.
5. Si se requiere un stall, ambas instrucciones en el paquete deben volverse stall.

- a) Analice en el código las dependencias de datos y determine cuales generan data hazards en nuestro procesador one-issue, **sin forwarding-stall**. En cada caso indique: el tipo de hazard, el operando en conflicto y los números de las instrucciones involucradas. Suponga que los saltos están perfectamente predichos, de modo que los *control hazard* son manejados por hardware.

Tipo dependencia	Reg	Instru	Hazard?
Raw-cond	X5	1-11	Si
Raw-cond	X5	1-3	No
Raw-cond	X5	1-10	No
Raw-cond	X5	1-10	No
Raw-cond	X5	10-11	Si
Raw-cond	X10	3-4	Si
Raw-cond	X10	3-8	No
Raw-cond	X11	4-5	Si
" "	X11	4-6	Si
Raw-cond	X12	5-7	Si
" "	X13	6-7	Si
Raw-cond	X14	7-9	Si
Raw-cond	X15	8-9	Si

```

1  ADD X5, XZR, XZR
2  B ENT
3  TOP: LSL X10, X5, #3
4  ADD X11, X1, X10
5  LDUR X12, [X11, #0]
6  LDUR X13, [X11, #8]
7  SUB X14, X12, X13
8  ADD X15, X2, X10
9  STUR X14, [X15, #0]
10 ADDI X5, X5, #2
11 ENT: CMP X5, X6
12 B.NE TOP

```

- b) Dibuje un diagrama de pipeline que muestre cómo se ejecuta el código LEGv8 dado anteriormente en el procesador de 2-issue con pipeline y forwarding-stall. (Suponga que sale del bucle después de dos iteraciones.)

```

for(i=0; i!=j; i+=2)
    b[i]=a[i]-a[i+1];

```

	I	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
ADD	1	IF	ID	EX	Mem	Wb										
	/	/	/	/	/	/										
B	2		IF	ID	EX	Mem	Wb									
		/	/	/	/	/	/									
CMP	11			IF	ID	EX	Mem	Wb								
				/	/	/	/	/								
STALL					/	/	/	/	/							
STALL					/	/	/	/	/							
B.NE						IF	ID	EX	Mem	Wb						
						/	/	/	/	/						

ES NECESARIO?

NO

Instruction	PC	Op	IF	ID	EX	Mem	WB
LSD	3	Loop	/	/	/	/	/
ADD	4		/	/	/	/	/
LDUR	5		/	/	/	/	/
ADD	8		/	/	/	/	/
LDUR	6		/	/	/	/	/
ADDI	10		/	/	/	/	/
STALL			/	/	/	/	/
STALL			/	/	/	/	/
SUB	7		/	/	/	/	/
STR	9		/	/	/	/	/
NOP			/	/	/	/	/
CMP			/	/	/	/	/
UOR			/	/	/	/	/
B.NE			/	/	/	/	/
UOR			/	/	/	/	/
Loop			/	/	/	/	/